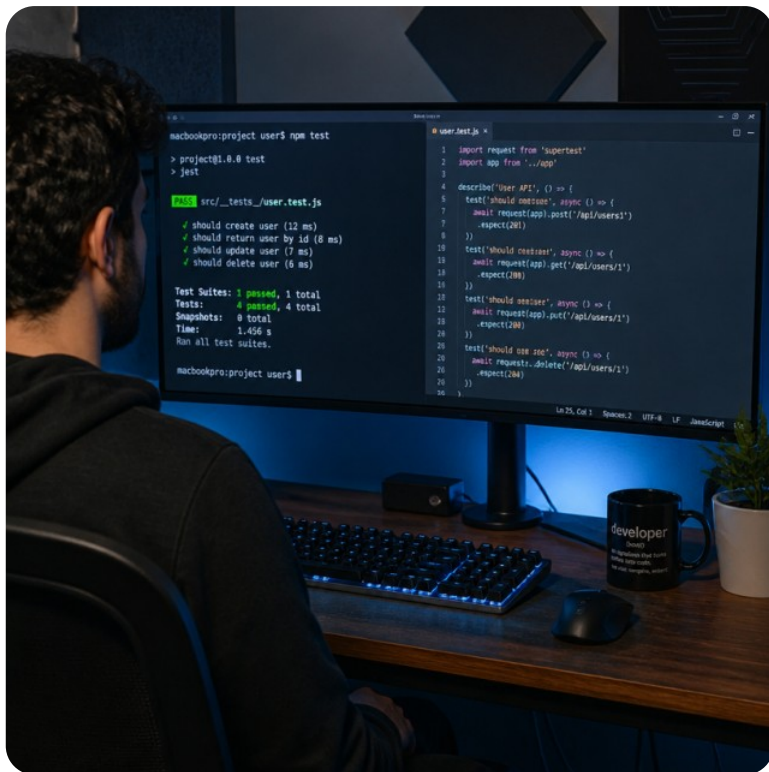


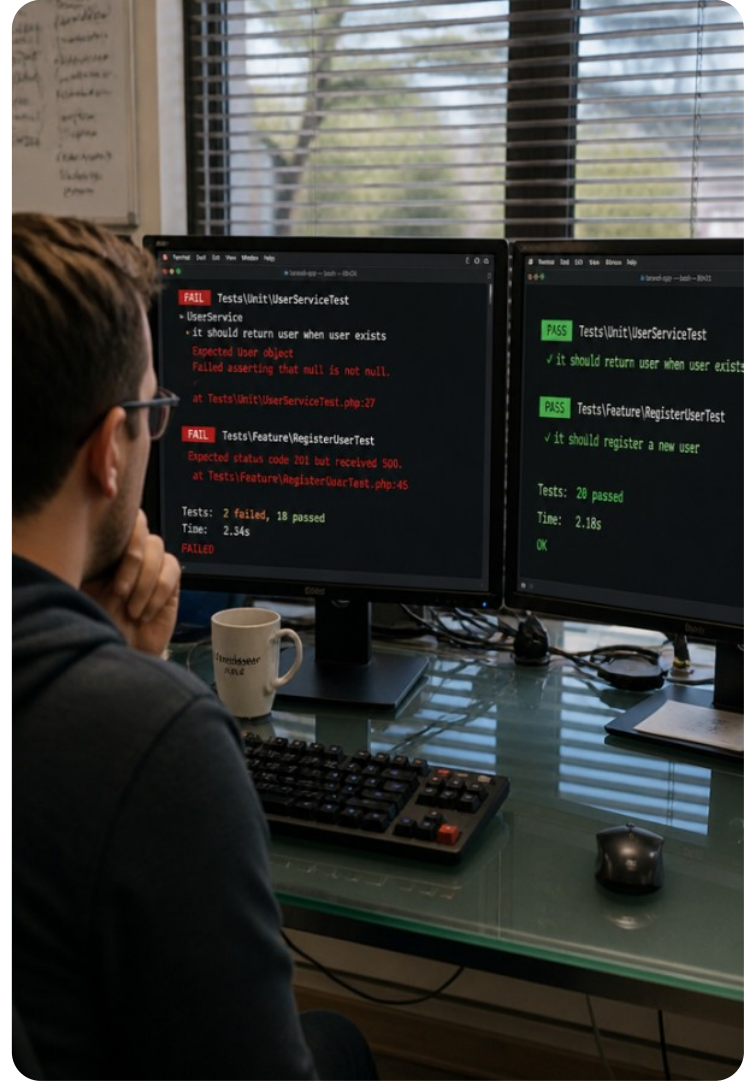


TDD e Debugging Moderno no Frontend

Construindo software resiliente com testes primeiro e depuração ágil.

E se o Teste Nascesse Antes?

Imagine nunca mais ter medo de refatorar código em produção.



Objetivos da Aula



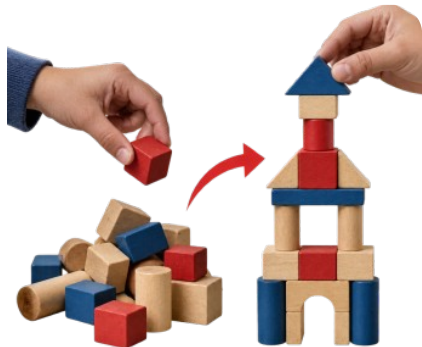
Neste bloco de aprendizagem, você desenvolverá competências essenciais para projetar código confiável desde o primeiro minuto:

Dominar o ciclo **Red-Green-Refactor** do TDD em cenários reais.

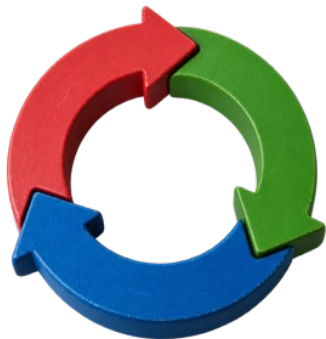
Aplicar estratégias de **debugging avançado** com breakpoints e modo watch.

- Discernir os cenários ideais de aplicação e trade-offs do TDD.

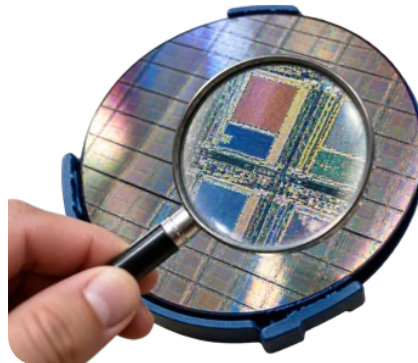
Vocabulário Fundamental



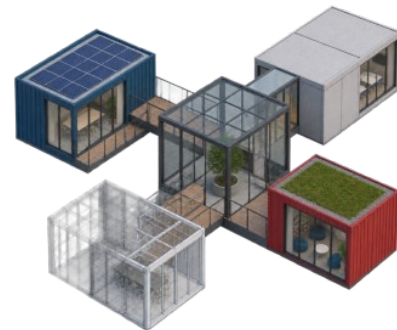
Refatoração -
Melhorar a
estrutura interna
do código sem
alterar seu
comportamento
externo.



Ciclo - Sequência
iterativa e
repetitiva de
passos
controlados para
guiar o design de
software.



Depuração -
Processo metódico
de rastrear, isolar e
remover erros
lógicos em
sistemas de
software.



Arquitetura -
Estrutura
organizacional dos
módulos e fluxos
de dados de uma
aplicação web.

Recapitulação: Cobertura de Código

REVISÃO

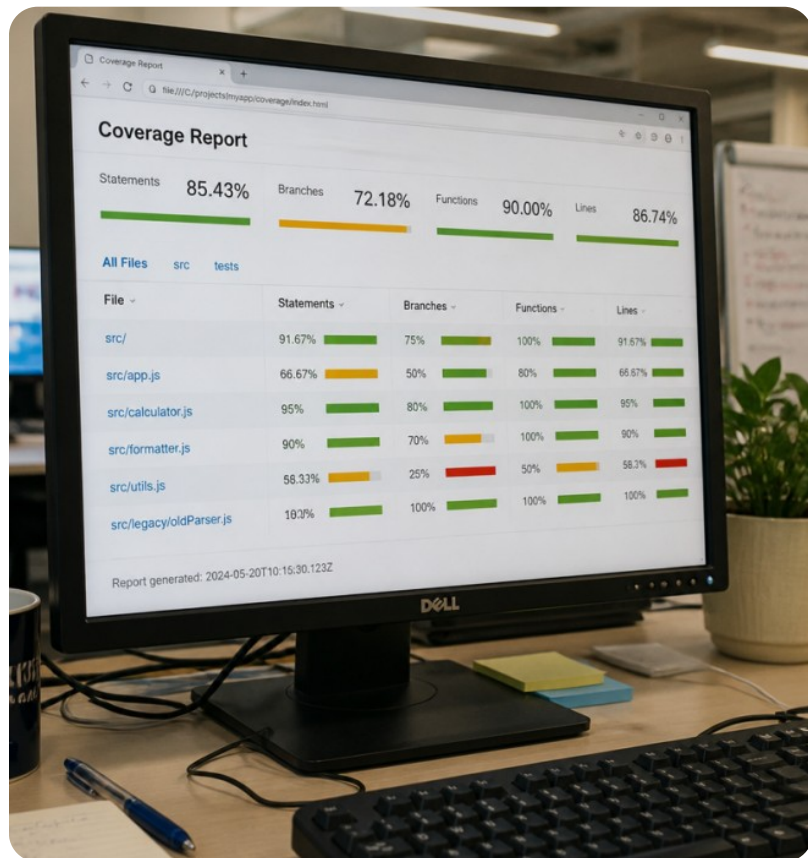
Analizamos relatórios do Istanbul/Vitest. A meta de **85%** equilibra qualidade e esforço.

- Cobertura alta não elimina falhas lógicas.
- Gaps em relatórios HTML evitam falsas certezas.



Ponto-chave

Cobertura aponta código não testado, não garante lógica validada.



Os Quatro Tipos de Cobertura

Relembre como cada métrica audita diferentes camadas do código-fonte executado:



Exemplo

Line & Statement: Verifica quais linhas físicas e instruções isoladas foram interpretadas pelo motor JavaScript durante a suíte de testes.



Atenção

Branch & Function: Mede se todas as ramificações de if/else e switch foram percorridas, e se cada função declarada foi de fato invocada.

Quiz: Interpretação de Cobertura



Pergunta 1:

Qual métrica garante que ambas as opções de uma condicional if/else foram executadas?

Pergunta 2:

Por que buscar 100% de cobertura frequentemente tem Retorno sobre Investimento (ROI) negativo?

Pergunta 3:

Se um arquivo possui 100% de Line Coverage, ele pode conter bugs em produção?

Quiz: Interpretação de Cobertura



Resposta 1:

Branch Coverage (cobertura de ramificação).

Resposta 2:

Porque o esforço para cobrir casos triviais ou código boilerplate consome muito tempo e traz pouco ganho real de confiabilidade.

Resposta 3:

Sim, pois a linha pode ser executada sem que os testes verifiquem valores limítrofes ou dados inválidos inesperados.

A Virada de Mentalidade

Desenvolver com TDD inverte o fluxo tradicional: primeiro você define o comportamento esperado como um cliente da API, depois constrói a solução.



O que é TDD?

CONCEITO

O **Test-Driven Development** é uma técnica de **design de software** de Kent Beck, não apenas uma conferência posterior.

- Guiado por iterações com feedback instantâneo.
- Força clareza sobre requisitos antes da lógica.



Lembre-se

O teste é o primeiro usuário do seu código, definindo nomes, parâmetros e retornos ergonômicos.

```
TS sum.ts  TS sum.test.ts X
1  import { describe, it, expect } from 'vitest'
2  import { sum } from './sum'
3
4  describe('sum', () => {
5      it('adds two positive numbers', () => {
6          expect(sum(1, 2)).toBe(3)
7      })
8
9      it('adds zero correctly', () => {
10         expect(sum(0, 5)).toBe(5)
11     })
12
13     it('handles negative numbers', () => {
14         expect(sum(-2, -3)).toBe(-5)
15     })
16
17     it('returns zero for opposite numbers', () => {
18         expect(sum(7, -7)).toBe(0)
19     })
20
21     it('works with large numbers', () => {
22         expect(sum(1000000, 2000000)).toBe(3000000)
23     })
24 })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

~/projects/tdd-sum

O Ciclo Red-Green-Refactor

O coração do TDD opera em um loop contínuo de três fases rigorosas e disciplinadas:

A diagram showing three red-outlined chevron shapes pointing to the right, connected in a sequence. The first chevron is on the left, the second in the middle, and the third on the right.

1. Red

Escreva um teste que expresse o requisito e veja-o falhar comprovadamente.

2. Green

Escreva o código mais simples possível para fazer o teste passar.

3. Refactor

Elimine duplicações e melhore a legibilidade mantendo a barra verde.

Fase Red: A Falha Deliberada

CICLO TDD

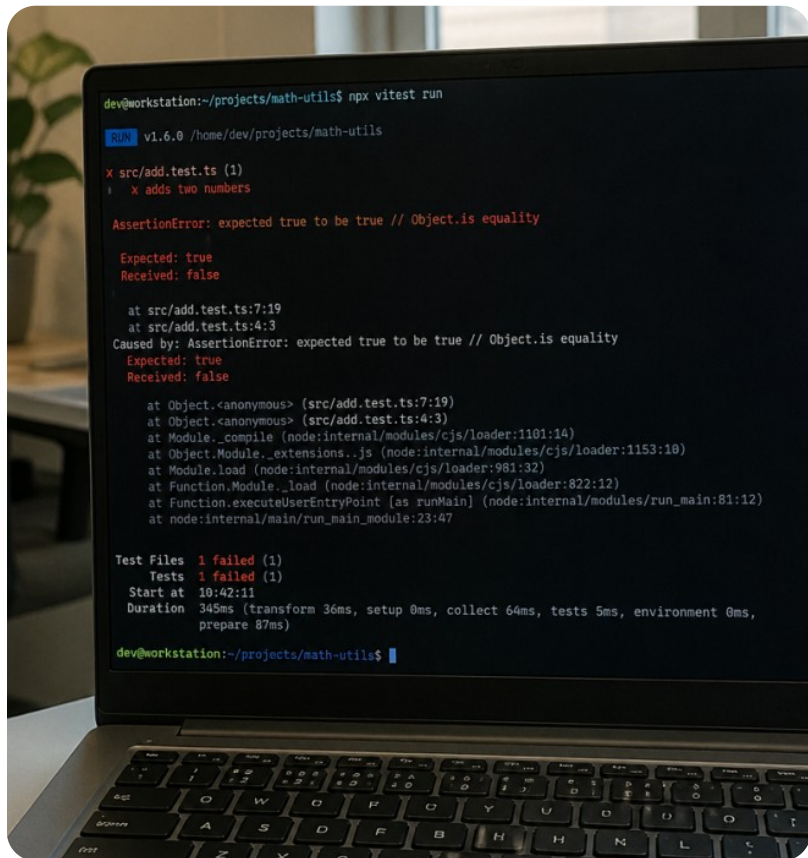
A fase **Red** valida se o teste funciona e não é um falso positivo.

- O teste deve falhar pelo motivo previsto (ex: função indefinida).
- Garante que a asserção é válida e o requisito é novo.



Atenção

Nunca implemente código de produção sem antes ver o teste falhar.



Fase Green: O Código Mínimo



```
JS state.js x
1 function getInitialState() {
2   return {
3     user: null,
4     todos: [],
5     filter: 'all'
6   };
7 }
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE

PASS test/state.test.js

- ✓ returns an object
- ✓ sets user to null
- ✓ sets todos to empty array
- ✓ sets filter to "all"

Test Suites: 1 passed, 1 total
Tests: 4 passed, 4 total
Snapshots: 0 total
Time: 0.328 s
Ran all test suites.

CICLO TDD

Na fase **Green**, a prioridade é fazer o teste passar rapidamente.

- Soluções literais são válidas (até retornar valores fixos).
- Evite over-engineering antes de consolidar os casos de teste.



Ponto-chave

Não programe cenários futuros; resolva estritamente a falha atual.

Fase Refactor: Limpeza Segura

CICLO TDD

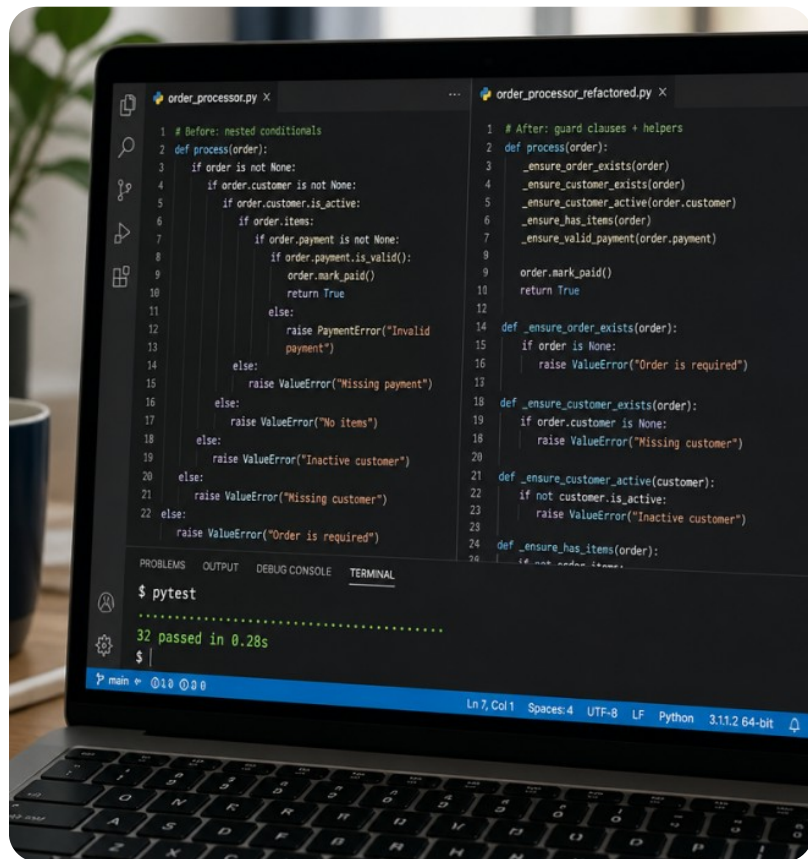
Com a barra verde assegurando que o comportamento está protegido, podemos aprimorar o código sem receio.

- Extrair variáveis, renomear identificadores e modularizar funções.
- Remover complexidade ciclomática e duplicações de lógica.



Lembre-se

Se um teste quebrar durante a refatoração, a rede de proteção funcionou imediatamente para alertá-lo.



Ordenação: Execução do Ciclo TDD

Ordene os passos sequenciais que compõem uma iteração correta de TDD:

Escrever um teste unitário que define a nova regra e vê-lo falhar.

Implementar o código mínimo estritamente necessário para obter sucesso.

Refatorar o código-fonte eliminando duplicações sem alterar comportamento.

Verificar a barra verde indicando que todos os testes foram atendidos.

Ordenação: Execução do Ciclo TDD

Ordene os passos sequenciais que compõem uma iteração correta de TDD:



1
Escrever um teste unitário que define a nova regra e vê-lo falhar.

2
Implementar o código mínimo estritamente necessário para obter sucesso.

3
Verificar a barra verde indicando que todos os testes foram atendidos.

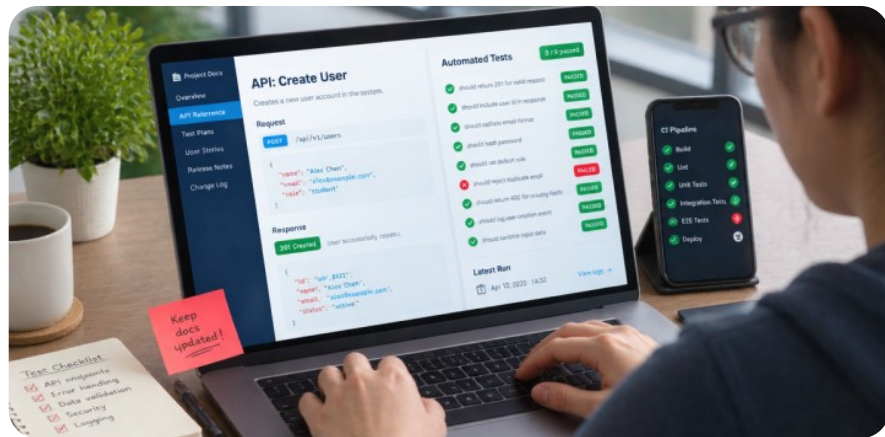
4
Refatorar o código-fonte eliminando duplicações sem alterar comportamento.

Vantagens Estratégicas do TDD



Código Testável por Design

Componentes desenvolvidos via TDD nascem com baixo acoplamento e alta coesão, pois módulos difíceis de testar geram atrito imediato no teste.



Documentação Viva e Atual

A suíte de testes descreve com precisão cirúrgica exatamente o que cada módulo faz, sem defasar como documentações em texto estático.

Mitos e Desafios no Frontend



ANÁLISE CRÍTICA

TDD no frontend enfrenta mitos e desafios:

Mito: 'TDD dobra o tempo'. Realidade: reduz retrabalho e debugging.

Desafio: Testar CSS com TDD é improdutivo; foque em lógica de estado e interação.



Atenção

Evite testar detalhes visuais; foque em comportamento, acessibilidade e regras de negócio.

Checagem: Filosofia do TDD

O TDD exige que você escreva previamente todos os testes da aplicação inteira antes de começar a codificar a primeira função.



🤔 *Prepare-se para explicar o seu raciocínio.*

Checagem: Filosofia do TDD

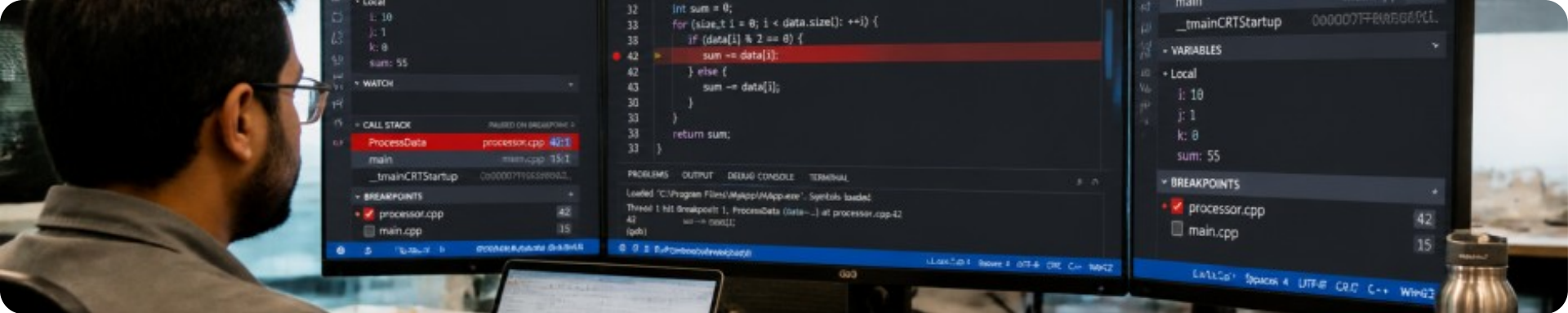


O TDD exige que você escreva previamente todos os testes da aplicação inteira antes de começar a codificar a primeira função.



Por que é isso?

O TDD opera em micro-ciclos iterativos: escreve-se um teste pequeno por vez, faz-se passar e refatora-se.



Estratégias de Depuração Moderna

DEBUGGING

Quando um teste quebra inesperadamente ou uma regra falha, o desenvolvedor precisa de ferramentas analíticas superiores ao simples uso de .

- Depuração metódica reduz horas de investigação frustrante a minutos de inspeção controlada.
- Dominar breakpoints e o ciclo de execução do Node/V8 acelera o diagnóstico de bugs sutis.

Breakpoints no VS Code

FERRAMENTAS

Utilizar **breakpoints** pausa o teste na linha desejada:

- Inspeção valores no painel de escopo (Variables Scope).
- Analise a pilha de chamadas (Call Stack) para ver a origem.
- Avalie expressões no Debug Console sem alterar o código.



Exemplo

Use a extensão do VS Code ou a flag `--inspect-brk` para debugar nativamente.



Modo Watch no Vitest

FERRAMENTAS

O Vitest monitora arquivos via grafo de dependências, reexecutando apenas testes impactados.

- Ciclo de feedback rápido durante o TDD.
- Filtro interativo de arquivos via atalhos no terminal.
- Mantém o foco sem alternar para o terminal.



Lembre-se

O comando roda em modo watch por padrão, reagindo a cada arquivo salvo.

```
alex@alex-laptop: ~/projects/todo-app x +
RUN v1.4.0 /home/alex/projects/todo-app
PASS src/utils/date.test.ts (8)
  ✓ formatDate formats date correctly (2)
  ✓ isToday returns true for today (1)
PASS src/components/ToDoItem.test.tsx (12)
  ✓ renders todo text (3)
  ✓ toggles completed state (2)
  ✓ deletes todo item (2)
PASS src/store/todoStore.test.ts (10)
  ✓ adds a todo (2)
  ✓ toggles a todo (1)
  ✓ removes a todo (1)
Test Files 3 passed (3)
Tests 9 passed (9)
Start at 10:21:35
Duration 412ms (transform 120ms, setup 0ms, collect 91ms, tests 201ms,
environment 0ms, prepare 0ms)

WATCH watching...

p update snapshot
u update snapshot (interactive)
f run failed tests
r rerun all tests
t filter by test name
p filter by pattern
q quit watch mode
h show help
>
```

Isolamento de Cenários com Vitest

Em suítes com centenas de testes, isolar o caso sob investigação é indispensável:



Ponto-chave

`test.only('cenário', () ⇒ {...})`: Foca exclusivamente neste teste ou bloco describe, ignorando todos os demais na suíte temporariamente.



Atenção

`test.skip('cenário', () ⇒ {...})`: Pula temporariamente um teste quebrado ou em construção enquanto você investiga outras partes do sistema.

Time-Travel Debugging em E2E

DEBUGGING

Ferramentas como Cypress/Playwright gravam cada evento do navegador:

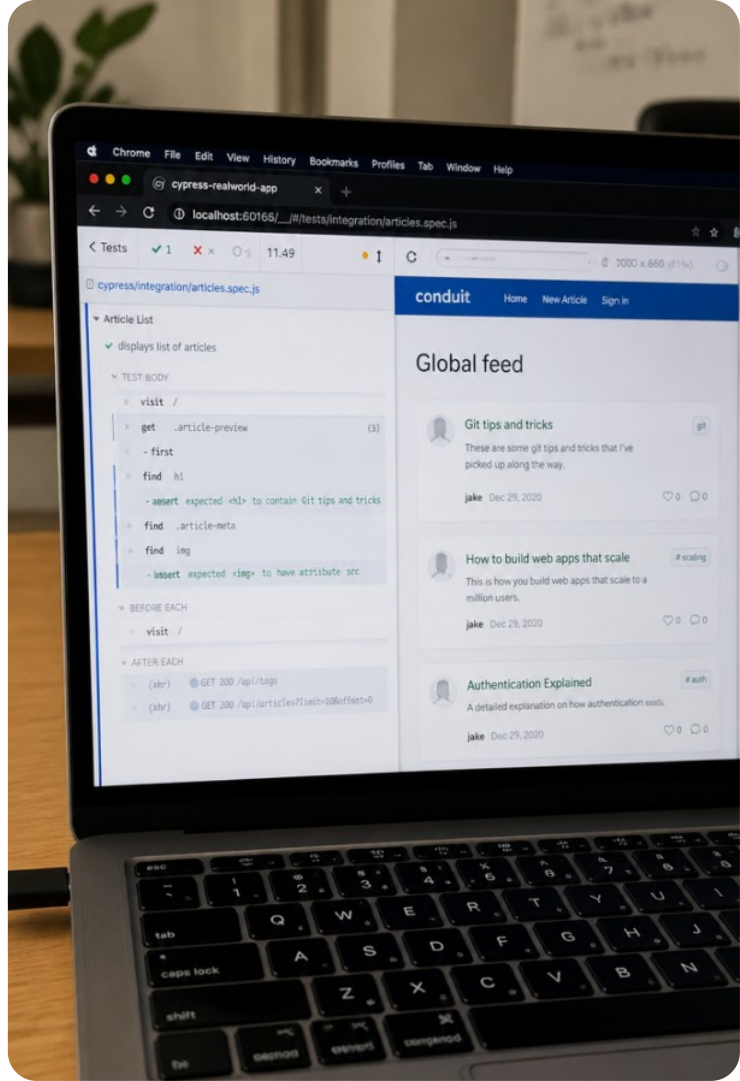
Time-Travel: Navegue pelo histórico e veja a UI antes/depois de cada ação.

Snapshots: Inspecione requisições HTTP e logs no exato milissegundo da falha.



Curiosidade

O time-travel elimina a necessidade de adivinhar estados que causam falhas intermitentes (flakiness).



TDD na Prática em Vídeo



Veja o ciclo Red-Green-Refactor no editor:



Prática 1: Validador de CPF via TDD

DESAFIO PRÁTICO

Vamos construir uma função robusta aplicando estritamente o ciclo TDD.

- O algoritmo do CPF brasileiro valida formato, dígitos repetidos e dois dígitos verificadores matemáticos.
- Começaremos pelo caso mais elementar e avançaremos progressivamente.

CPF Passo 1: O Teste Vazio (Red)

CICLO TDD

Iniciamos com o caso nulo ou inválido mais imediato no arquivo de teste:

Ao executar, o Vitest falha com .



Ponto-chave

Temos nosso primeiro teste Red legítimo confirmando que a função nem sequer existe.

```
student@devbox:~/projects/registration-form$ npm test
> registration-form@1.0.0 test
> jest

FAIL tests/validation.test.js
  ● Validation › should reject invalid email

  ReferenceError: validateEmail is not defined

    at Object.<anonymous> (tests/validation.test.js:12:3)
    at Promise.then.completed (node_modules/jest-circus/build/utils.js:298:28)
    at new Promise (<anonymous>)
    at callAsyncCircusFn (node_modules/jest-circus/build/utils.js:231:10)
    at _callCircusTest (node_modules/jest-circus/build/run.js:316:40)
    at _runTest (node_modules/jest-circus/build/run.js:252:3)
    at _runTestsForDescribeBlock (node_modules/jest-circus/build/run.js:126:9)
    at run (node_modules/jest-circus/build/run.js:71:3)
    at runAndTransformResultsToJestFormat (node_modules/jest-circus/build/legacy-co...

Test Suites: 1 failed, 1 total
Tests:      1 failed, 1 total
Snapshots:  0 total
Time:       0.512 s
Ran all test suites.

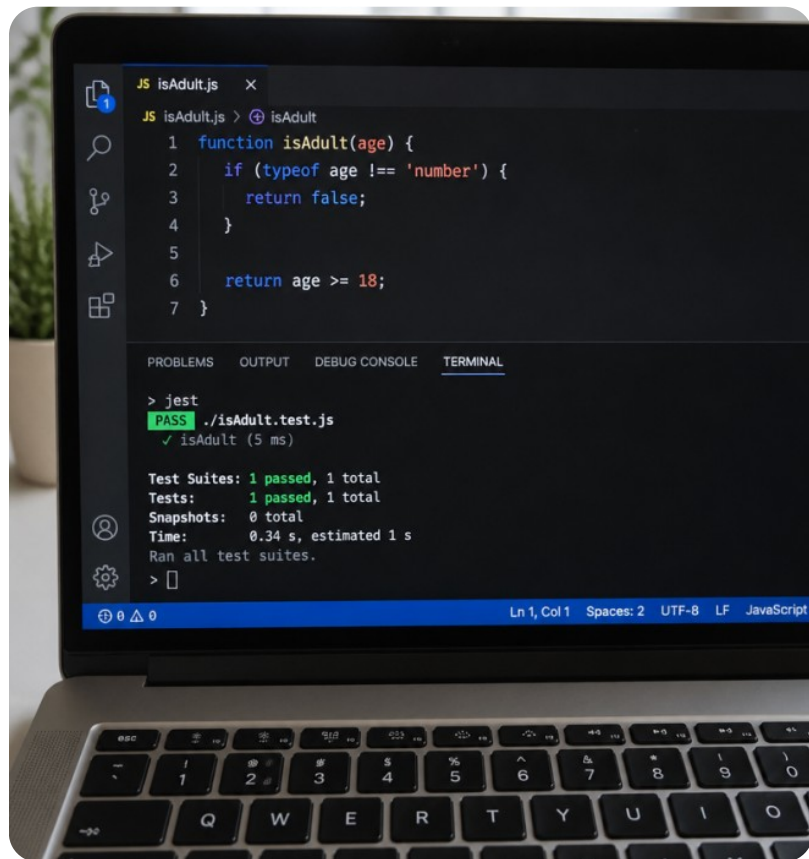
npm ERR! Test failed. See above for more details.
```

CPF Passo 2: O Verde Mínimo (Green)

CICLO TDD

Criamos a implementação mínima no arquivo :

O teste agora passa instantaneamente com barra verde no terminal.



```
JS isAdult.js X
JS isAdult.js > isAdult
1 function isAdult(age) {
2   if (typeof age !== 'number') {
3     return false;
4   }
5
6   return age >= 18;
7 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> jest
PASS ./isAdult.test.js
✓ isAdult (5 ms)

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 0.34 s, estimated 1 s
Ran all test suites.
> |
```



Lembre-se

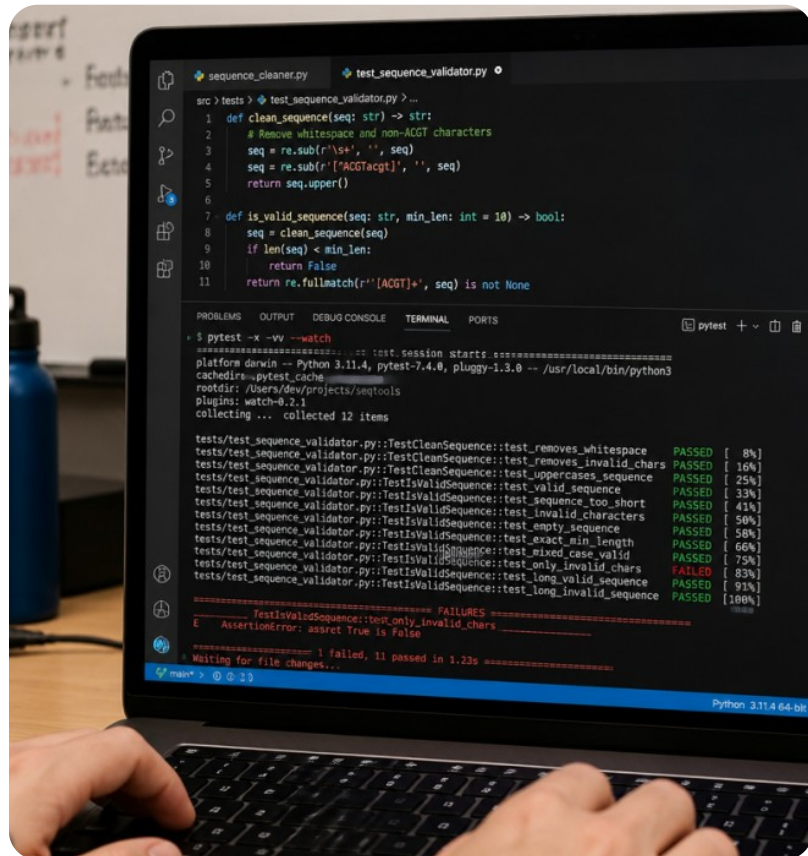
Ainda não implementamos cálculo matemático complexo; resolvemos somente o que o teste exigiu.

CPF Passo 3: Tamanho e Dígitos Iguais

CICLO TDD

Adicionamos novos testes para CPFs com formatos incorretos e dígitos repetidos como '111.111.111-11':

O teste falha (Red), e então implementamos regex de limpeza e validação de sequência repetida (Green).



CPF Passo 4: Dígitos Verificadores

Finalmente, testamos o cálculo matemático dos dois dígitos verificadores reais:



Exemplo

Novo Caso de Teste (Red):

```
expect(validarCPF('52998224725')).  
  toBe(true);  
  
expect(validarCPF('52998224726')).  
  toBe(false);
```



Lembre-se

Refatoração (Refactor):

Extraímos o algoritmo de soma ponderada e módulo 11 em uma função utilitária pura interna .

Prática 2: Filtro Dinâmico no Frontend

DESAFIO PRÁTICO

Agora aplicaremos o TDD para um componente de interface: um campo de busca que filtra uma lista de produtos por título e categoria.

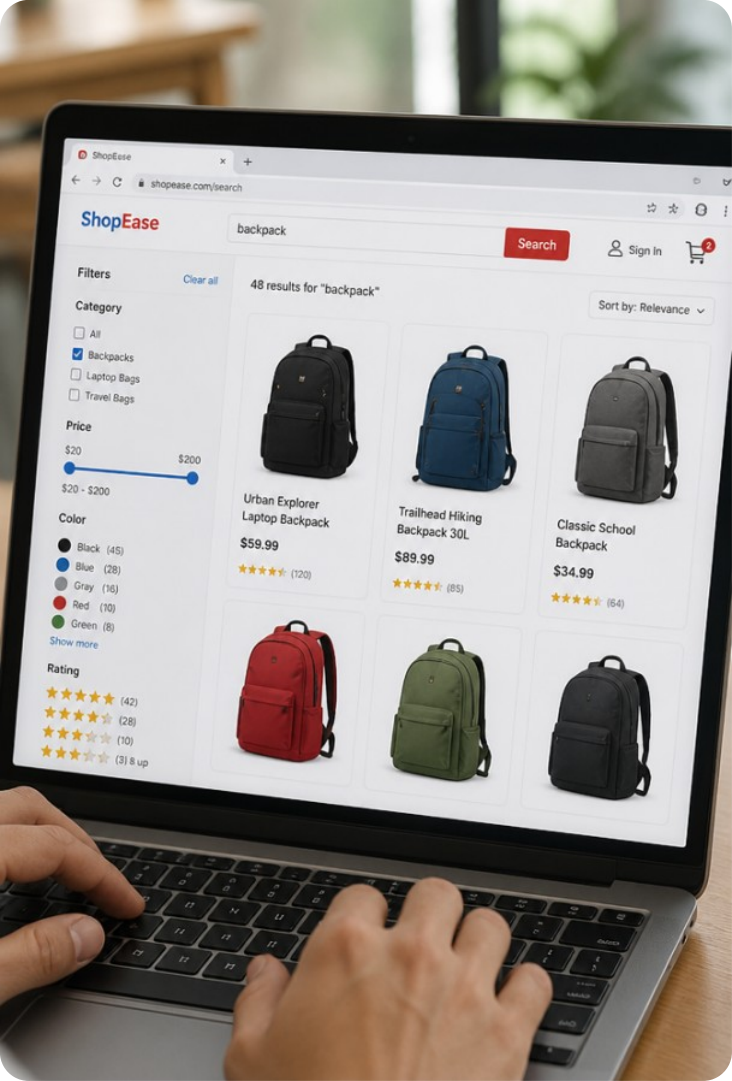
Utilizaremos ou DOM puro com Vitest.

- O teste simula o comportamento de digitação do usuário e valida os nós visíveis.



Ponto-chave

Não testamos o estado interno do componente; testamos o que o usuário enxerga após digitar.



Ciclo do Filtro de Produtos



1. Teste Red

Renderizar lista com 3 itens e disparar digitação 'Teclado'; esperar 1 item visível.



2. Código Green

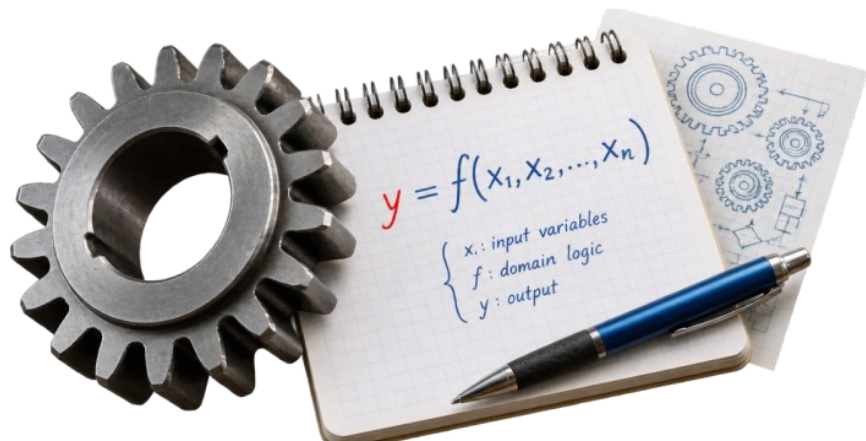
Implementar input com event listener básico e filtro simples.



3. Refactor Seguro

Adicionar debounce para performance e normalização de maiúsculas/acentos sem quebrar o teste.

Quando Usar e Quando Evitar TDD?



Exemplo

Quando Adotar TDD: Regras de negócio complexas, cálculos, validações de formulário, parsers de dados e utilitários de alta criticidade.



Atenção

Quando Evitar TDD: Prototipação rápida (PoCs), ajustes puramente estéticos de CSS e exploração de bibliotecas gráficas desconhecidas.

Associação de Conceitos e Ferramentas



Relacione as palavras com as definições

Red-Green-Refactor

Método do Vitest para executar exclusivamente um cenário específico sob depuração.

Time-Travel

Recurso do Cypress/Playwright que permite navegar pelos estados do DOM gravados em cada etapa.

Breakpoint

Instrução do depurador no VS Code que pausa a execução na linha indicada para inspeção.

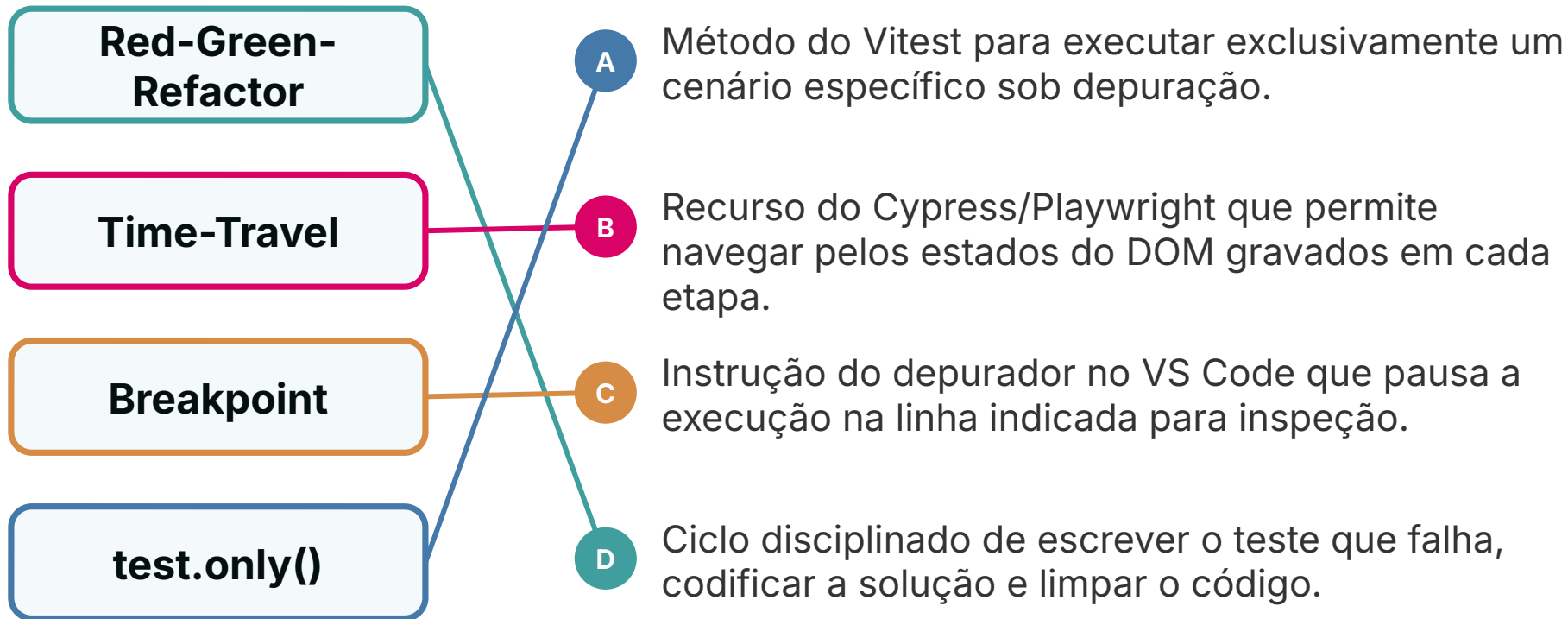
test.only()

Ciclo disciplinado de escrever o teste que falha, codificar a solução e limpar o código.

Associação de Conceitos e Ferramentas



Relacione as palavras com as definições



Debate: O Custo de Começar com Testes



Muitos times alegam que adotar TDD em projetos de startups desacelera a entrega inicial de funcionalidades. Sob o ponto de vista de arquitetura e custos de longo prazo, esse argumento se sustenta?

Debate: O Custo de Começar com Testes



Você poderia ter dito...

A favor do TDD: O tempo na concepção reduz bugs em produção, refatorações emergenciais e custos de suporte.

Contra o TDD em PoC: Se requisitos mudam diariamente, testes prévios podem ser descartados em excesso.

Síntese dos Aprendizados

REVISÃO

Consolidamos competências essenciais:

TDD: molda o design antes da implementação.

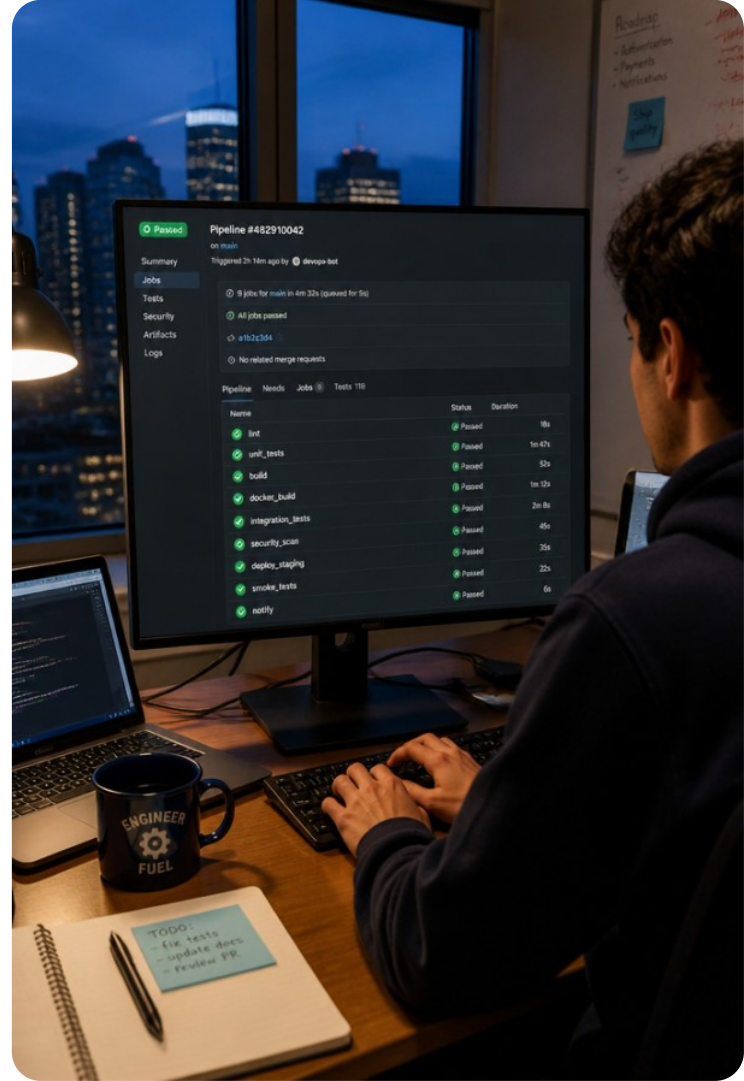
Ciclo Red-Green-Refactor: valida requisitos, satisfaz necessidades e garante longevidade.

Depuração: breakpoints e modo watch substituem intuição por observação metódica.



Lembre-se

Próxima aula: integraremos testes à esteira de CI/CD com GitHub Actions.



Avaliação Final: O Ciclo TDD

Qual é a principal justificativa técnica para exigir que um teste falhe (fase Red) antes de escrever o código de produção correspondente?

1. Calcular a métrica de cobertura de código no relatório Istanbul com maior velocidade.
2. Forçar o navegador a reiniciar a compilação do bundle JavaScript no Vite.
3. Garantir que a documentação HTML do projeto seja gerada automaticamente na nuvem.
4. Comprovar que o teste é sensível à funcionalidade e não passa por um falso positivo.

Avaliação Final: O Ciclo TDD



Qual é a principal justificativa técnica para exigir que um teste falhe (fase Red) antes de escrever o código de produção correspondente?

1. Calcular a métrica de cobertura de código no relatório Istanbul com maior velocidade.
2. Forçar o navegador a reiniciar a compilação do bundle JavaScript no Vite.
3. Garantir que a documentação HTML do projeto seja gerada automaticamente na nuvem.
4. Comprovar que o teste é sensível à funcionalidade e não passa por um falso positivo.